

GSLT2GSLT: Typed Action Decoding over Verified Interfaces

A machine-checked waist for neural program synthesis, and the belief-state architecture it licenses

Zarathustra A. Goertzel Oruži

July 2026 — working draft

Abstract

We describe an architecture for neural program synthesis over generalized syntax-semantics language theories (GSLTs) in which the interface between learning and logic is machine-checked. The network never emits program text: it scores *typed construction actions* over an exact elaboration state, and a checker owns legality, completion, and acceptance. Three layers are formalized in Lean 4 with kernel-checked proofs: the discrete skeleton (legality masks proved sound, complete, and free of dead ends), a *trust boundary* (learned signals may reorder search but provably cannot lose solutions; hard pruning is licensed exactly by per-property recall certificates, with a machine-checked counterexample showing a plausible uncertified mask destroying a genuine solution), and a *decoder-state theory* (recurrent workspaces as damped fixed-point iteration, with a finite optimal settling depth, a parallel-scan correctness license, and a belief-state cell whose fusion rule is Bayesian conditioning). A single lemma family shows Kalman updating, gated recurrent mixing, and probabilistic-logic evidence revision to be one operation—addition of evidence in natural coordinates, seen from the moment chart. Empirically, the resulting typed-action decoder replicates Alien-Coding-style cumulative self-improvement on OEIS program discovery, with every increment exactly verified; a twelve-cell factorial disentangles decoder architecture from auxiliary supervision; and the same interface, pointed at an external dependent-type benchmark, *measured* the benchmark’s definitional-equality demands ($\beta\eta$) through a graded ladder of verified roots. We state the open comparison the theory sharpens: whether building Bayesian fusion into the decoder beats learning Bayes-shaped mixing, and where—by theorem—learning is necessary because the world does not hold still.

1 Introduction

Neural program synthesis systems typically trust a language model to emit well-formed programs and filter failures afterwards. We invert the trust: the target language—a GSLT, given as data—induces an exact elaboration state with typed holes; the network only *ranks legal refinements*; the checker computes what each refinement entails and decides acceptance. The scientific benefit is attributability. When the interface between proposer and checker is itself proved correct, a discovery cannot be an artifact of a malformed candidate, and a failure cannot hide an interface bug: three times in the development reported here, an apparent neural failure was traced by the formal layer to an interface boundary (a mask dead-end, a defaulted intermediate type, a missing conversion rule) *before* any training run could mis-attribute it to learning.

The architecture is a waist (Figure 1): one small action vocabulary through which all learning speaks. Everything above the waist (encoders, decoder state, ranking, search order) is free;

everything below it (legality, spine construction, acceptance) is checked; and the boundary itself is a pair of theorems rather than a convention.

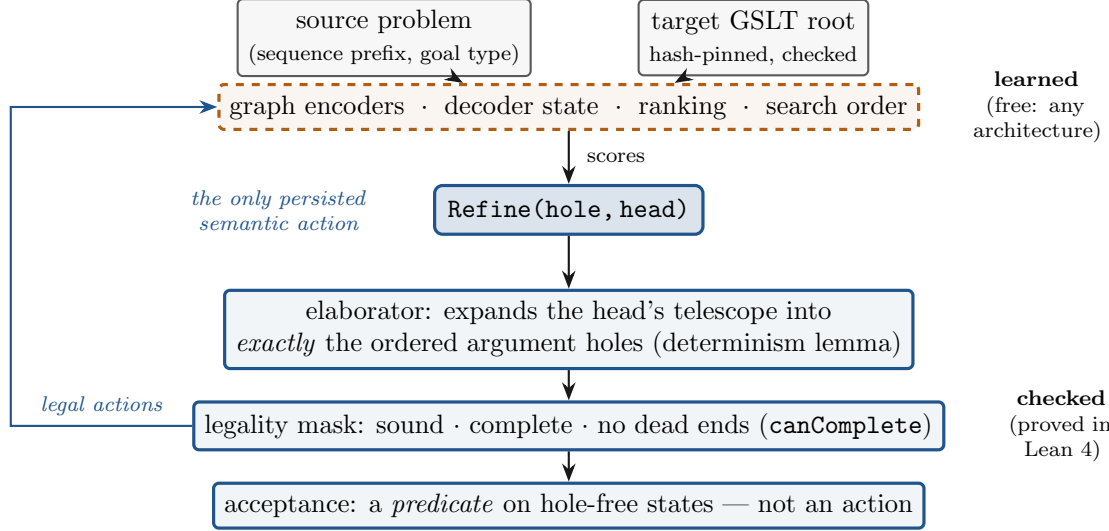


Figure 1: The waist. All learning speaks through one action vocabulary; the checker owns what each action entails and whether a state is accepted. The narrow interface is what makes architecture comparisons attributable: nothing above the waist can widen what is acceptable below it.

2 The waist: `Refine(hole, head)`

The persisted semantic action is the pair *refine this typed hole with this head*. Dependent argument holes are not actions: applying a head expands, deterministically, into exactly the ordered argument holes its telescope dictates (a determinism lemma, not a convention). Termination is not an action: acceptance is a predicate on states with no open holes. A policy may factor its score as $P(\text{hole} \mid s) P(\text{head} \mid s, \text{hole})$, but the interface persists the pair.

For the first-order root (a 16-operator combinator DSL for integer-sequence programs, after [1]), the legality automaton is specified and sealed in Lean: soundness (accepted streams parse to well-formed programs), completeness (every well-formed program within budget is reachable with every prefix legal), and *no dead ends*—from every reachable legal state some completion exists within budget (**canComplete**). The completion relation is exported with provenance: the operator table and mask fixtures that the Python trainer consumes are generated from the Lean development and hash-pinned, so the learned system and the proved system share one source of truth. The same atomic interface is then instantiated for a dependent typed-hole root (Section 6), with a proved equivalence to the sealed first-order automaton as the non-vacuity gate: if the generic interface could not express the sealed machine exactly, the interface would be wrong.

3 The trust boundary

Two theorems, proved once over the root-parametric interface, fix what learned signals may do.

Theorem 1 (Soft ranking is always safe). *Any external ranking that lists exactly the legal actions—in any order, from any scorer, at any parameters—preserves the accepted language. Learned guidance can reorder search; it cannot lose or manufacture solutions.*

Theorem 2 (Hard pruning is licensed exactly by certificates). *A pruning mask derived from a property certified for the target under the evaluation semantics never prunes a state from which a target-reproducing completion exists (**certifiedMask-recall-preserving**, lifted from completed programs to partial search states). Sign, parity, and modular-residue certificates instantiate the schema.*

The pair is kept honest by a mandatory counterexample: a plausible, uncertified parity-of-root mask is exhibited, machine-checked, pruning the genuine constant-one solution (**oddRootHardMask-prunes_oneSolution**). Uncertified evidence—including everything a world model merely *believes*—receives soft authority only; a world-model hook converts a certified proposition into a licensed mask, and an empty evidence slot fails the premise. This is the design doctrine “the model proposes, the checker disposes” upgraded to mathematics, with the failure mode it prevents exhibited rather than asserted. Figure 2 is the whole boundary on one page.

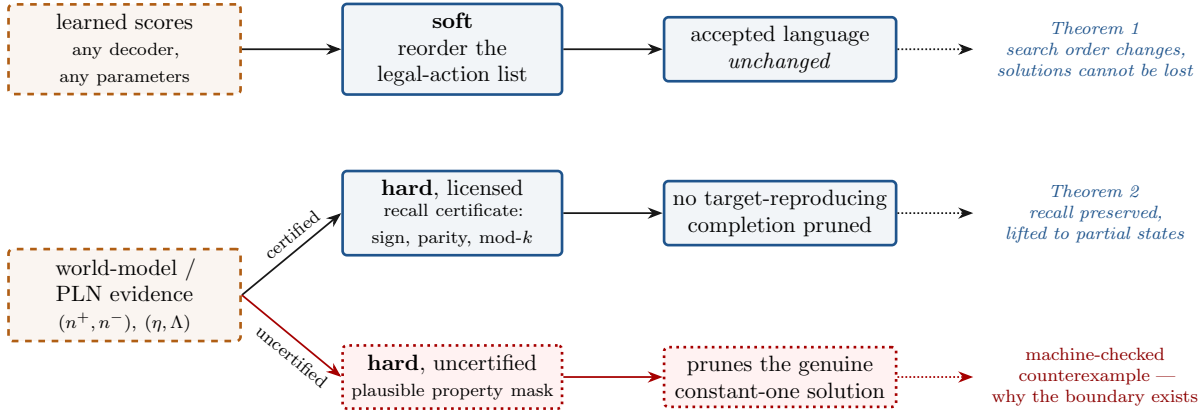


Figure 2: The trust boundary. Learned signals may always reorder; they may exclude only behind a certificate. The red branch is not a caution but a theorem: a plausible uncertified mask provably destroys a real solution.

4 Decoder state: from one gated vector to a belief workspace

The incumbent decoder carries construction history in a single gated recurrent vector. Its natural generalization is a *workspace*: slots with fixed addresses and evolving contents, updated by reusable read–transform–gate–write operators

$$w_i \leftarrow w_i + \frac{1}{K} \sum_{k=1}^K g_{ki} (c_{ki} - w_i), \quad (1)$$

an architecture family we adapt from an unpublished recurrent-workspace design (personal communication, 2026), itself descended from external-memory networks and shared-workspace models [5]. Our adaptation types the addresses: slots are indexed by the typed holes of the

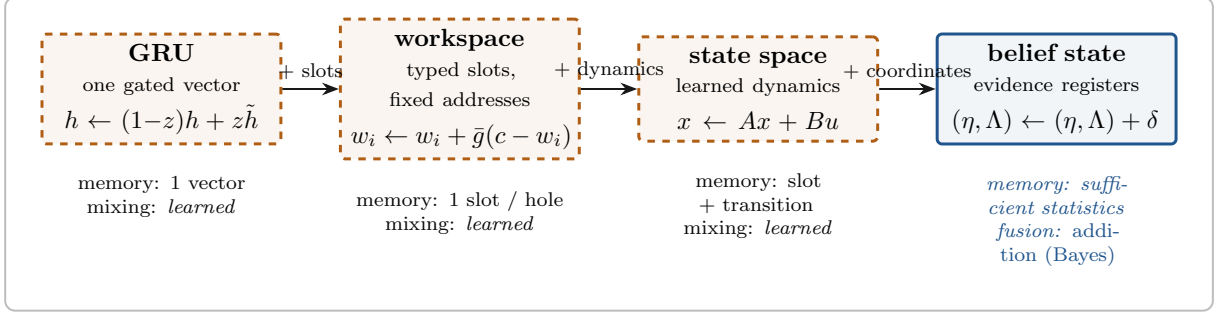


Figure 3: The decoder-state ladder. Each rung adds structure and gives away freedom; only the last replaces learned mixing with the addition Bayes prescribes. The trust boundary holds identically along the whole ladder, so architecture is a free variable.

elaboration state, so the workspace is a typed register file on which the program graph grows, and slot allocation is total and injective by the sealed open-hole bound.

The formal scope of this family is developed in a dedicated Lean tree (`WorkspaceDecoder`, Figure 3): update (1) is damped fixed-point iteration; under a spectral condition the linear model contracts to a unique equilibrium; and under finite-depth training mismatch there is a *unique fitted recurrence depth*, with over-settling strictly worse—so depth is a convergence parameter with an optimum, not a capacity knob. A prefix-scan theorem (affine transitions form a monoid; scanned prefixes equal the sequential trajectory) is the correctness license for parallelizable linear-recurrent variants [7]. A bridge through the implicit function theorem licenses equilibrium differentiation [6] where the contraction condition holds. Crucially, the trust boundary of Section 3 is inherited: for *any* workspace parameters, gates, and depth, acceptance and recall are unchanged—the entire architecture axis lives on the safe side by construction.

4.1 Fusion is addition; charts are shadows

The crown of the state theory is a single identity. Write belief in *natural coordinates*: evidence counts (n^+, n^-) in the discrete case [8], the information form $(\eta, \Lambda) = (\Sigma^{-1}\mu, \Sigma^{-1})$ in the Gaussian case. Then:

- Bayesian fusion of independent evidence is *addition* in natural coordinates—componentwise for counts, matrix addition for information;
- the Kalman update, the gated-interpolation step (1), and probabilistic-logic revision are the *same* addition, viewed after normalizing to the moment chart; the optimal gate is the Kalman gain, and the workspace equilibrium under exact gains is the Bayesian posterior mean (`equilibrium_iff_eq_posteriorMean`);
- strength and confidence are derived columns of the counts, recovered by the sealed confidence characterization; the counts are the sufficient statistics and the primal state.

The identity is a commuting square (Figure 4): fusion is addition upstairs; every gated-interpolation rule anyone has ever hand-designed is that same addition seen downstairs, after normalizing to the moment chart.

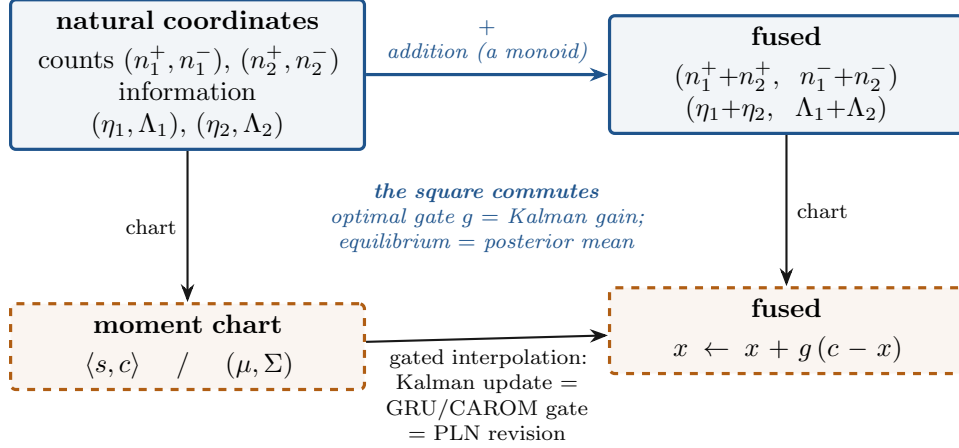


Figure 4: Fusion is addition; charts are shadows. Three rules invented independently in three literatures — Kalman filtering, gated recurrence, probabilistic-logic revision — are one operation, and it is only nonlinear because the moment chart is.

Sequentially, Riccati covariance propagation makes the optimal gain time-varying, and a sharp condition ($R_2(P + R_1) \neq R_1^2$ in the scalar case) implies every *constant* gate is strictly suboptimal—the derived reason input-conditioned gating (“selectivity”) earns its complexity.

This licenses a concrete cell (Figure 5): slots hold additive evidence registers; learned operators propose *measurements* (what observations mean); a learned gain network, conditioned on the slot’s own precision, scales them; the state update is pure addition; readout charts to moments and scores only legal actions. Everything Bayes determines (how to fuse) is hardwired; everything Bayes leaves open (what observations mean, what the noise is) is learned.

4.2 Where hardwiring is provably wrong: the open world

The optimality of hardwired addition is conditional, and each condition fails somewhere we care about. Addition never forgets: confidence is monotone in accumulated evidence. Under drift—a world whose parameters move—monotone accumulation is *provably* suboptimal; the optimal filter fades memory, and the correct decay rate is derived from the drift statistics, not tuned. Naive addition double-counts correlated re-observation. A single natural-coordinate slot projects away multimodal posteriors. And a miscalibrated measurement model poisons exact fusion, where a learned mixer could have compensated.

We take an explicit position: *optimizing for the closed world is the wrong default*. Worlds drift; open-world probabilistic-logic systems build forgetting in for good reason. Mathematical domains—the OEIS loop below, proof search—are the distinguished near-stationary regimes where hardwired fusion may legitimately excel, and that is precisely why they are the right first battleground: if structure cannot win *there*, it wins nowhere.

The boundary theory is now sealed, and each failure mode is a theorem with a formula. *Nesting*: precision addition realizes exactly the interpolation gates $[0, 1)$ of the learned-mixing family, and the unit-gate complement splits cleanly into observation discount, strict old-evidence decay, and overwrite—the precise freedoms hardwired fusion lacks. *Double counting*: naive addition overstates precision by exactly the declared evidence overlap, in closed form, and a discounted fusion (proved also count-natively) restores calibration. *Drift*: in the jump model

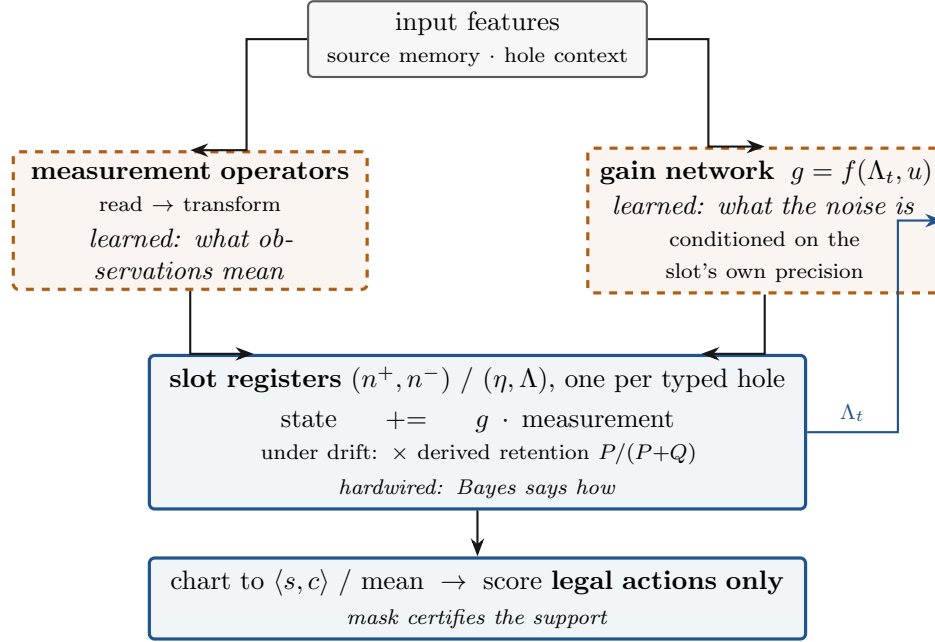


Figure 5: The cell the theorems license. Learning concentrates where Bayes is silent — in the measurement maps and the gain schedule — while fusion itself is the addition Bayes prescribes. The precision register feeds the gain network, which is what makes selectivity possible; the readout can only rank actions the checker already admits.

$P \mapsto P + Q$, every accumulator whose effective variance stays $\leq P$ under-gains and carries strictly greater risk whenever $Q > 0$; the derived old-information retention is $P/(P + Q)$, moment retention $R/(P + Q + R)$, repeated updates fade exponentially, and $Q = 0$ recovers stationary addition—forgetting as a *derived rate*, not an aesthetic. *Miscalibration*: for a distorted measurement $y = dx + \varepsilon$, learned mixing strictly beats the uncorrected unit-model rule exactly off the boundary $(d - 1)(Pd - R) = 0$; but recalibrating the measurement map ($y \mapsto y/d$, $R \mapsto R/d^2$) reproduces the same optimal update with fusion left hardwired—and against the *true-model* gain, learned mixing cannot win at all, since that gain is already optimal. Every fusion-stage repair thus relocates to the measurement map.

The design conclusion is not “hardwire everything” but *learning is necessary, and theorems say where*: in the measurement maps, in the gain schedules, and—under drift—in a decay whose rate the jump statistics determine, while fusion itself remains addition wherever stationarity survives contact with the domain. Each theorem doubles as a pre-registered diagnostic: whether a learned arm’s advantage comes from forgetting, observation discount, dependence repair, or measurement recalibration is identifiable from the per-slot trajectories the experiment schema already carries. One implementation obligation follows: fractional forgetting requires an honest weighted-evidence layer—it is not to be smuggled into natural-number counts.

4.3 Routing, order, and the switching boundary

The workspace family has a natural command-driven refinement (personal communication, 2026): a small vocabulary of opaque command tokens, softmax-routed over K reusable read–transform–gate–write experts, with slot state split into an immutable evidence field, a fixed positional code,

and the evolving contents that update (1) governs. Four boundary theorems now frame what routing can and cannot add.

Routing stays on the ladder. A simplex mixture of gated-interpolation experts is again a single gated interpolation under the mixed gain (`mixedExpertStep_eq_singleInterpolation`), so routing buys input-conditioned selectivity—which the Riccati argument above already priced as necessary—without leaving the update class; and the immutable evidence field survives every routing schedule, temperature, and depth (`runTripleSlotSchedule_evidence`). Command tokens themselves carry no semantics: trajectories are determined by the induced routing schedule alone (`commandTrajectory_eq_of_mixtureSchedule_eq`), and distinct tokens with equal routes are behaviorally indistinguishable. Meaning lives in routes, not names.

Compositionality is commutation. When are two command phases order-independent, so that a schedule is a *set* of edits rather than a brittle sequence? For affine phases the answer is exact: swapping the order shifts the state by $[B, A]x + (Ba + b - Ab - a)$ (`affineCompositionDiscrepancy_exact`), and order-independence holds iff the linear parts commute *and* the affine obstruction vanishes (`affinePhases_orderIndependent_iff`); commuting linear parts alone are not enough—a one-dimensional bias counterexample closes that loophole. Equivalently, the pairwise interference energy $\|[A, B]\|_F^2$ must vanish (`linearPhases_orderIndependent_iff_interferenceEnergy_zero`)—a Gram diagnostic computable from any trained operator pair. Compositionality thereby stops being an emergent hope and becomes a measured certificate.

Stability does not compose. Two commands, each individually extinguishing—either alone reaches rest in two steps—alternate to 4^n growth (`individuallyStable_switchingDiverges`). Per-command guarantees are therefore worthless for command *programs*; the honest certificate is schedule-uniform. A common quadratic Lyapunov function bounds every schedule at one geometric rate, and commuting stable generators always admit one (`commutingCommonQuadraticLyapunov_crown`). A command-driven decoder should ship with a switching certificate, not a per-command one.

Safety is indifferent to all of it. The trust boundary extends verbatim: for arbitrary routes, temperatures, schedules, and depths, ranked acceptance coincides with checker acceptance and recall of accepted programs is unchanged (`inheritanceCrown`). The routing axis, like the state axis, lives entirely on the safe side; the theorems above are about competence, not safety.

When trained operators cannot be made to commute, there is a second, older route to order-independence: canonicalize. Interleave the learned step with a projection onto canonical forms, $s_{t+1} = \text{canon}(f_\phi(s_t, c_t))$, and order-sensitivity is quotiented away rather than trained away: a confluent canonicalizer sends both orders of a critical pair to one normal form—exactly the commutation the raw operators lack—while the per-step projection stops representational drift from compounding across a schedule. The iterated form is a weight-tied loop of the same transformer, the regime in which looped transformers act as programmable computers [11]; canonicalization is the projection that keeps the loop’s state on the quotient. This program already runs on that principle at two other scales: the determinism lemma of Section 2 is the confluence statement for spine creation, and canonical-forms LF makes conversion vanish into the syntax. The design conclusion repeats itself one level up: learn the proposal, hardwire the quotient.

5 The loop compounds, and the factorial that kept us honest

The empirical instance is cumulative self-improvement on OEIS program discovery [1] (Figure 6): train on the accumulated verified corpus, search under fixed budgets, check every candidate against the whole encyclopedia, add what verifies, repeat. The same wake-sleep shape drives other self-improving synthesis systems, which additionally distil reusable abstractions into the library between rounds [10]; here the checker plays the role of the sleep-phase filter, and the waist fixes what a reusable action is.

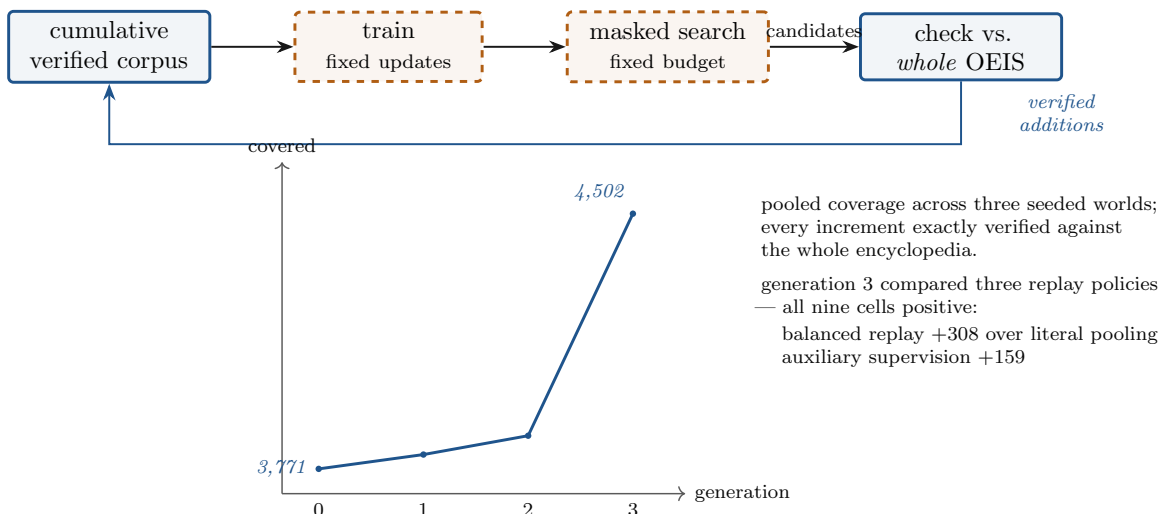


Figure 6: The loop compounds. At literal pooling the model’s own discoveries carry 0.24% of sampling mass and the loop starves on the signal it exists to generate; balanced replay is what makes self-improvement visible.

Three independently seeded worlds grew coverage from a shared initial 3,771 known sequences to 3,986 after two generations, and—under a prospectively registered three-way comparison of replay policies at generation three—to 4,502 before generation four, with all nine cells positive. Balanced replay of the model’s own discoveries (25% mass) beat literal pooling by +308 pooled sequences: at natural sampling mass (0.24%), the loop starves on exactly the signal it exists to generate. Auxiliary predictive supervision added +159. The union of generation-three discoveries was 503 distinct sequences, for 697 verified model-origin programs accumulated by generation four.

Two registered negatives bound the claims. An ablation asking whether a world’s own discoveries beat an equal mass of *unused external* certified data answers no—but that comparator does not exist in the deployed regime, where the corpus is whatever the loop has verified; it is an oracle-advantaged ablation, not the loop objective. And no arm, in any generation, solved its own assigned held-out target within budget: the evidence concerns learned *discovery* guidance over the catalogue, not target-conditioned synthesis.

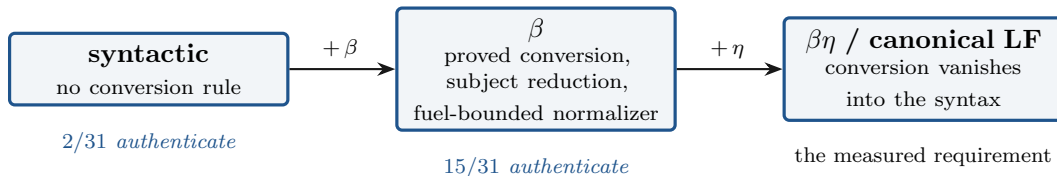
A twelve-cell factorial (decoder architecture $\times$ auxiliary supervision $\times$ three seeds) disentangled an earlier confound: a one-seed comparison had suggested a token-sequence decoder beat the typed-action decoder 38–28; the factorial showed the apparent gap was carried by asymmetric auxiliary supervision, and across seeds the typed-action decoder led pooled (188 vs. 119, positive in 2/3 seeds) with the supervision interaction inconsistent—no interaction claim

survives. One-seed architecture stories, in this regime, are noise until replicated.

The registered comparison the state theory now sharpens is three-lineage: the gated recurrent incumbent, the typed workspace (learned mixing, CAROM-style), and the belief-state cell (hard-wired fusion)—matched capacity, identical trunks warm-started and decoder-specific parameters reinitialized, two further cumulative generations, per-slot evidence trajectories logged in a pinned schema, and the interpretation bins written before launch. A parallel 2×2 tests a shallow predictive-coding residual adapter over *both* backbone architectures; the interaction contrast is the estimand, and the settling theory predicts the workspace backbone is where a local settling rule has its best chance.

6 The same waist, pointed outward

Because the interface is root-parametric, pointing it at a new language is an instantiation, not a rewrite. Pointed at an external dependent-type inhabitation benchmark [2], the authentication gate became a measuring instrument for the benchmark itself (Figure 7):



the 16 residuals have unequal β -normal forms but equal $\beta\eta$ -normal forms — so checking this benchmark’s witnesses requires full $\beta\eta$ definitional equality, a fact its own selection criteria do not state. Discovered because the gate refused to blur syntactic equality into conversion.

Figure 7: A graded ladder of verified roots turns the authentication gate into an instrument: each rung is a checker of strictly greater conversion strength, and the witnesses that fail measure exactly what the benchmark assumes. The ladder doubles as an ablation axis — one policy, evaluated under checkers of increasing power.

under a conversion-free root, 2/31 reference witnesses authenticate; adding proved β -conversion (with subject reduction and a fuel-bounded normalizer whose exhaustion is a classified verdict, never acceptance) raises this to 15/31; the sixteen residuals have unequal β -normal forms but equal $\beta\eta$ -normal forms—so checking the benchmark’s witnesses requires full $\beta\eta$ definitional equality, a fact its own “no definitional reduction” selection criterion does not reveal, discovered because the gate refused to blur syntactic equality into conversion. All 31 statements round-trip through the typed interface, and 782 named corruption fixtures (including 596 binder-capture cases) are rejected. The graded ladder of verified roots—syntactic $\subset \beta \subset$ canonical-forms LF —doubles as an ablation axis unavailable elsewhere: the same policy can be evaluated under checkers of increasing conversion strength.

A frozen one-shot under the pre-conversion root recorded the honest baseline: a ranker trained only on independent synthetic data solved 0/31 within budget while simple controls solved up to 3 and a native solver [2] solved 31/31—the root and action representation, not neural capacity, are the binding constraint before conversion lands, and the result is banked as the pre-conversion floor for the ladder. Beyond dependent types, a read-only audit of the Metamath database family

found thousands of non-copied proofs across related databases, making database-to-database transfer a measurable next rung; and a generated-calculus benchmark waits, gated, behind its checker’s readiness theorem.

7 Related work and provenance

The proposer/checker split with learned guidance descends from neural theorem proving and self-learning synthesis [1]; typed-hole elaboration states from live programming semantics [3] and bidirectional elaboration [4]; the workspace family from shared-workspace and external-memory architectures [5] and equilibrium models [6]; selective state-space models [7] supply the parallelizable linear-recurrent variants our scan theorem licenses; evidence-count semantics from probabilistic logic networks [8]; the external dependent-type benchmark and solver from [2]. The contribution here is the machine-checked *waist*: masks, trust boundary, and decoder-state theory proved in Lean 4 with kernel-checked axioms confined to `{propext, Classical.choice, Quot.sound}`, fixtures generated from the proofs, and the learned system bound to them by hash-pinned parity — so that architecture comparisons are attributable, and failures indict the component that actually failed.

Status

Sealed and machine-checked: the first-order skeleton and its exports; the atomic interface with two proved root instances; the trust-boundary pair with its counterexample; the workspace scope (contraction, fitted depth, scan license, IFT bridge); the sequential and matrix belief theory; the counts-primal unification; the selective belief-decoder contract and its pinned schema; and the fusion boundary theory of Section 4.2 (expressivity nesting, overlap calibration, nonstationary fading, distortion factorization—four lifts, fifteen new theorem families, five explicit scope boundaries, kernel-computed accounting). Registered and running or queued: generation-four portfolio cells; the predictive-coding adapter gate; workspace depth calibration; the three-lineage comparison. All coverage numbers above are exact whole-corpus verifications from immutable, independently recomputed ledgers.

References

- [1] T. Gauthier and J. Urban. *Learning Program Synthesis for Integer Sequences from Scratch*. AAAI 2023; arXiv:2202.11908.
- [2] C. Norman and J. Avigad. *Implementing Dependent Type Theory Inhabitation and Unification*. arXiv:2603.01463, 2026.
- [3] C. Omar, I. Voysey, M. Hilton, J. Aldrich, and M. Hammer. *Hazelnut: A Bidirectionally Typed Structure Editor Calculus*. POPL 2017.
- [4] L. de Moura, J. Avigad, S. Kong, and C. Roux. *Elaboration in Dependent Type Theory*. arXiv:1505.04324, 2015.
- [5] A. Goyal et al. *Coordination Among Neural Modules Through a Shared Global Workspace*. ICLR 2022; arXiv:2103.01197.

- [6] S. Bai, J. Z. Kolter, and V. Koltun. *Deep Equilibrium Models*. NeurIPS 2019.
- [7] A. Gu and T. Dao. *Mamba: Linear-Time Sequence Modeling with Selective State Spaces*. arXiv:2312.00752, 2023.
- [8] B. Goertzel, M. Iklé, I. Goertzel, and A. Heljakka. *Probabilistic Logic Networks*. Springer, 2009.
- [9] K. Cho et al. *Learning Phrase Representations using RNN Encoder–Decoder for Statistical Machine Translation*. EMNLP 2014.
- [10] K. Ellis et al. *DreamCoder: Bootstrapping Inductive Program Synthesis with Wake–Sleep Library Learning*. PLDI 2021; arXiv:2006.08381.
- [11] A. Giannou et al. *Looped Transformers as Programmable Computers*. ICML 2023; arXiv:2301.13196.